Daily Paper Review: BYOD — Build Your Own DLM: Efficient Autoregressive-to-Diffusion Language Model Conversion

Auto-generated Report

November 3, 2026

1 Paper Summary

1.1 Problem and Motivation

Diffusion language models (DLMs) offer attractive properties over autoregressive (AR) models—parallel token generation, flexible infilling, and bidirectional attention—but training them from scratch is prohibitively expensive. LLaDA 8B required approximately 130,000 GPU-hours; DiffuLLaMA required approximately 1,917 GPU-hours with 65B tokens of continued pre-training. Meanwhile, a vast ecosystem of high-quality instruction-tuned AR checkpoints exists freely. Kuiper and Bagheri [1] ask whether an existing AR model can be cheaply retrofitted into a functioning masked diffusion generator, and propose a recipe that accomplishes this in just 3–4 GPU-hours on a single consumer GPU.

1.2 Method

BYOD applies two architectural modifications to a frozen instruction-tuned AR model: (1) removal of the causal attention mask, enabling bidirectional conditioning; and (2) attachment of rank-1024 LoRA adapters (α=1024, dropout=0) to the query and value projections only. All other parameters—keys, MLPs, normalization, embeddings, and the language modeling head—are frozen. This yields 4–6% trainable parameters depending on the base model.

Rather than expanding the vocabulary, each model reuses an existing single-token string from its tokenizer as the mask token (e.g., `MASK` for Gemma/Llama/Qwen, `<?>` for Ministral), avoiding any embedding layer modification.

Training objective. Training uses the absorbing-mask discrete diffusion framework (as in LLaDA/MDLM). Given a clean sequence $\mathbf{x}_0$, corruption time $t \sim \mathcal{U}[10^{-3}, 1]$ is sampled, and each answer-region token is independently masked with probability t, producing $\mathbf{x}_t$. The prompt region is never masked. The per-region reconstruction loss is:

$$\ell_R(\theta) = -\frac{1}{t \cdot |R|} \sum_{i \in \mathcal{M}_t \cap R} \log p_{\phi,\theta}(x_0^i \mid \mathbf{p}, \mathbf{x}_t), \tag{1}$$

where R is a region (answer A or padding P), $\mathcal{M}_t$ is the set of masked positions, $\mathbf{p}$ is the prompt, ϕ denotes frozen base parameters, and θ denotes LoRA parameters. The $1/t$ factor normalizes gradient scales across noise levels. The combined loss weights answer and padding regions 90:10:

$$\mathcal{L}(\theta) = \mathbb{E}_{t,\mathbf{x}_0,\mathbf{x}_t}\big[0.9\,\ell_A(\theta) + 0.1\,\ell_P(\theta)\big]. \tag{2}$$

Training data. A 1-million-example mixture with fixed 256-token sequence length, comprising general instruction data (45%, from Clean-Instruct-3M, Tulu 3, Alpaca-GPT4, Alpaca), multiple-choice reasoning (18%, MMLU, HellaSwag, ARC-Easy), mathematics (18%, Orca-Math, GSM8K), and code (19%, OpenCoder, MBPP). Total tokens processed: at most 102.4M (25,000 iterations $\times$ batch 16 $\times$ 256 tokens). Training uses AdamW with peak learning rate 10^{-5}, cosine decay, FP8 mixed precision, and gradient norm clipping at 0.5.

Inference. Generation starts with the prompt visible and the answer region fully masked. At each denoising step, the model predicts logits for all masked positions simultaneously; the most confident predictions are retained (unmasked) while lower-confidence positions are remasked. A delayed EOS mechanism assigns minimal retention confidence to end-of-sequence tokens, preventing premature termination. The process runs for a task-specific number of forward evaluations (NFE): 64 for most benchmarks, 256 for math and code tasks.

The authors convert four instruction-tuned models: Gemma 2 9B IT, Llama 3.1 8B Instruct, Qwen2.5 7B Instruct, and Ministral 8B Instruct.

1.3 Results

Across a nine-task benchmark suite (ARC-C, GPQA, HellaSwag, MMLU, MMLU-Pro, GSM8K, MATH-500, HumanEval, MBPP), BYOD models retain 70–85% of their AR parent's average accuracy at a cost of approximately 3.2 GPU-hours ($11):

- **BYOD-Gemma** achieves 50.6% average accuracy vs. 65.7% for Gemma AR, approaching LLaDA's 51.9% (trained from scratch at $\sim$130,000 GPU-hours).
- On **factual/reasoning** tasks (ARC-C, HellaSwag, MMLU), BYOD models *match or exceed* AR parents in several cases—BYOD-Gemma scores 0.752 on HellaSwag vs. 0.696 for Gemma AR.
- The largest degradation occurs in **math and code**: GSM8K drops from 0.872 to 0.640 (Gemma), and HumanEval from 0.740 to 0.256, reflecting the difficulty of non-sequential generation for tasks requiring strict left-to-right reasoning.
- **Code syntax failures** are a major issue: 28.0% parser failure rate on HumanEval and 8.8% on MBPP, primarily indentation and bracket-matching errors from parallel token prediction.

Open-ended generation quality (128-token completions scored by Phi-4) shows a larger gap: BYOD-Gemma achieves perplexity 4.33 at 128 NFE vs. 2.79 for LLaDA and 2.2–3.0 for AR parents. Lexical diversity (Distinct-1) trails AR baselines by 10–15 percentage points.

Adapter merging. Folding LoRA weights into base projections ($\mathbf{W}_{\text{merged}} = \mathbf{W} + (\alpha/r)\,\mathbf{BA}$) is nearly lossless: benchmark accuracy drops by only 0.4 points on average while perplexity actually improves by 2–3%, enabling deployment without adapter overhead.

Cost comparison. BYOD requires 3.2 GPU-hours ($11 on RTX Pro 6000) vs. 1,917 GPU-hours ($10,000) for DiffuLLaMA and 130,000 GPU-hours ($675,000) for LLaDA—a **600$\times$ to 40,000$\times$** cost reduction.

1.4 Limitations

The 256-token answer cap severely limits applicability to long-form generation and complex reasoning chains. The LoRA rank (1024), target projections (Q/V only), loss weighting (90:10), and data mixture ratios are all fixed without ablation, making it impossible to assess recipe robustness. Bidirectional attention quality varies dramatically across base models (100% exact-match copy for Gemma vs. 20% for Qwen) with no explanation. Training data overlaps with evaluation benchmarks (MMLU, HellaSwag, ARC-Easy, GSM8K, MBPP), complicating interpretation. No human evaluation is conducted, and the external perplexity scorer (Phi-4) reflects one model's preferences. Extended training (50k steps) yields only +0.5 points, suggesting LoRA capacity saturation without higher-rank or broader-target configurations being explored.

1.5 Relevance to My Research

This paper directly addresses **diffusion LLMs** (Topic 4) by establishing the efficiency frontier for AR-to-DLM conversion. The core finding—that LoRA-only adaptation with causal mask removal suffices to produce functional masked diffusion generators—has implications for the broader diffusion LLM ecosystem: it means every new AR release can be immediately converted into a diffusion variant at negligible cost. The 70–85% accuracy retention with only 102.4M training tokens suggests that the "knowledge gap" between AR and diffusion paradigms is smaller than previously assumed—most of the AR model's capabilities transfer through simple architectural modification, with the main deficit in sequential reasoning tasks. The code syntax failure analysis (28% on HumanEval) provides concrete evidence for where non-sequential generation fundamentally struggles, informing future work on hybrid AR-diffusion architectures.

2 Follow-up Research Directions

2.1 Direction 1: Structured-Output-Aware Denoising for Code Generation

Gap: BYOD's 28% parser failure rate on HumanEval stems from parallel token prediction violating syntactic constraints (indentation, bracket matching, scope nesting). Standard confidence-based unmasking treats all token positions as independent, ignoring the tree-structured dependencies inherent in code.

Approach: Introduce a *syntax-guided unmasking schedule* that respects abstract syntax tree (AST) constraints. During denoising: (1) predict all masked tokens simultaneously as usual; (2) before accepting predictions, run an incremental parser on the partially-revealed sequence to identify positions where acceptance would create a syntax error; (3) force-remask those positions regardless of confidence, allowing them to be re-predicted conditioned on syntactically-valid neighbors in subsequent steps. The incremental parser (e.g., tree-sitter) operates in $O(n)$ per step. Additionally, train with a *structure-aware noise schedule* that preferentially masks syntactically-dependent tokens together (e.g., matching brackets, indentation blocks), teaching the model to predict them jointly. This requires modifying the forward corruption process to use dependency-correlated masking rather than independent per-token masking.

Feasibility: High. Tree-sitter parsing adds minimal overhead per denoising step (<1ms for 256-token sequences). The correlated masking modification requires only changes to the data pipeline, not the model architecture. The main risk is that constraint enforcement may reduce generation diversity by overly restricting the denoising trajectory.

2.2 Direction 2: Progressive Rank Expansion for Closing the Quality Gap

Gap: BYOD uses fixed rank-1024 LoRA and shows signs of capacity saturation (only +0.5 points from doubling training steps). The 2.7–5× perplexity gap vs. AR parents suggests

that Q/V-only rank-1024 adapters are insufficient to fully adapt the frozen transformer to bidirectional diffusion generation. Yet full fine-tuning would eliminate the cost advantage.

Approach: Implement a *progressive rank expansion* schedule inspired by progressive growing in GANs. Start with rank-64 LoRA on Q/V projections (Phase 1, 5k steps), then expand to rank-256 on Q/K/V (Phase 2, 10k steps), then rank-1024 on Q/K/V/MLP-up/MLP-down (Phase 3, 10k steps). At each expansion, the existing low-rank factors are preserved and new dimensions are zero-initialized, maintaining continuity. The learning rate resets with warm-up at each phase transition. This progressive approach has two advantages: (1) early phases learn coarse bidirectional attention patterns cheaply, while later phases refine generation quality with more capacity; (2) the total trainable parameter count grows gradually, avoiding the optimization instability of initializing a very large adapter from scratch. Measure the quality-vs.-compute Pareto frontier against fixed-rank baselines at matched total FLOPs.

Feasibility: Moderate. The implementation requires managing LoRA configurations across phases but uses standard PEFT libraries. The main uncertainty is whether progressive expansion genuinely outperforms simply training a larger fixed-rank adapter for the same total compute, which requires careful controlled experiments. Memory constraints may limit the final-phase adapter size on consumer GPUs.

2.3 Direction 3: Hybrid AR-Diffusion Decoding with Paradigm Routing

Gap: BYOD demonstrates that factual/reasoning tasks transfer well to diffusion (some even improve over AR), while sequential tasks (math, code) degrade severely. This suggests that the optimal generation paradigm is task-dependent—neither pure AR nor pure diffusion is universally best. Yet no mechanism exists to dynamically choose the paradigm within a single model.

Approach: Since BYOD preserves the frozen AR weights and adds LoRA adapters, the *same checkpoint* can operate in both AR mode (with causal mask, LoRA disabled) and diffusion mode (bidirectional mask, LoRA enabled). Design a lightweight *paradigm router*: a small classifier (e.g., a linear probe on the prompt's [CLS] embedding) trained to predict which paradigm achieves higher quality for a given input. At inference time, the router selects AR or diffusion mode before generation begins. For a more fine-grained approach, allow *intra-sequence paradigm switching*: generate an initial draft via diffusion (fast, parallel), then identify low-confidence subsequences and regenerate them autoregressively (sequential, precise). The merged-adapter variant enables this without maintaining separate weight copies—simply apply or skip the adapter residual per mode.

Feasibility: High. The router is a simple binary classifier trainable on a few thousand (prompt, preferred-paradigm) pairs generated by comparing AR and diffusion outputs on a held-out set. Intra-sequence switching requires engineering the inference loop to support mixed attention masks but involves no model changes. The main risk is router accuracy—misrouting a code task to diffusion mode would yield high failure rates, so conservative routing (defaulting to AR for ambiguous cases) is advisable.

3 Conclusion

Kuiper and Bagheri demonstrate that converting instruction-tuned AR models into masked diffusion generators is feasible at remarkably low cost: 3.2 GPU-hours and $11 per model, achieving 70–85% of AR benchmark accuracy. BYOD-Gemma (50.6% average) approaches LLaDA (51.9%) at 40,000× lower cost, establishing a new efficiency frontier for diffusion LLM creation. The recipe is simple—remove the causal mask, attach rank-1024 Q/V LoRA adapters, and train on 102.4M tokens with the standard absorbing-mask objective. The key insight is that most of an AR model's knowledge transfers through this minimal adaptation; the main

deficits are in sequential reasoning (math, code) where non-sequential generation fundamentally struggles. The nearly lossless adapter merging enables deployment without overhead. However, the 256-token cap, lack of ablations on critical hyperparameters, substantial perplexity gap in open-ended generation, and 28% code syntax failure rate indicate significant room for improvement. The work opens a practical pathway for the diffusion LLM community: rather than training from scratch, future DLMs can bootstrap from the best available AR checkpoints at negligible marginal cost.

References

[1] Ruurd J.A. Kuiper and Robert A. Bagheri. BYOD: Build Your Own DLM — Efficient Autoregressive-to-Diffusion Language Model Conversion. *arXiv preprint arXiv:2610.41574*, 2026.